

Modelagem e Desenvolvimento Orientado a Objetos

Projeto: Academia — Planos e Frequência

1. Contextualização

O sistema proposto tem como objetivo auxiliar uma academia no controle de seus alunos, planos e frequência.

O sistema deverá permitir o cadastro e gerenciamento dos alunos, associar cada aluno a um plano e registrar seus check-ins. Uma das principais regras do sistema é que um aluno não poderá realizar um check-in caso seu plano esteja vencido ou inativo.

Para esta etapa da modelagem, foram selecionadas as classes `Aluno`, `Plano` e `CheckIn` como principais classes do domínio.

2. Classes do domínio

2.1 Classe Aluno

Representa uma pessoa matriculada na academia.

****Atributos:****

- * `nome: String`
- * `cpf: String`
- * `plano: Plano`

****Operações:****

- * `realizarCheckIn(checkIn: CheckIn): boolean`
- * `consultarPlano(): Plano`

2.2 Classe Plano

Representa o plano contratado pelo aluno e seu estado atual.

****Atributos:****

- * `tipo: String`
- * `valor: double`
- * `status: String`

****Operações:****

```
* `estaAtivo(): boolean`  
* `renovar(): void`
```

2.3 Classe CheckIn

Representa o registro de uma visita do aluno à academia.

****Atributos:****

```
* `dataHora: LocalDateTime`  
* `aluno: Aluno`  
* `liberado: boolean`
```

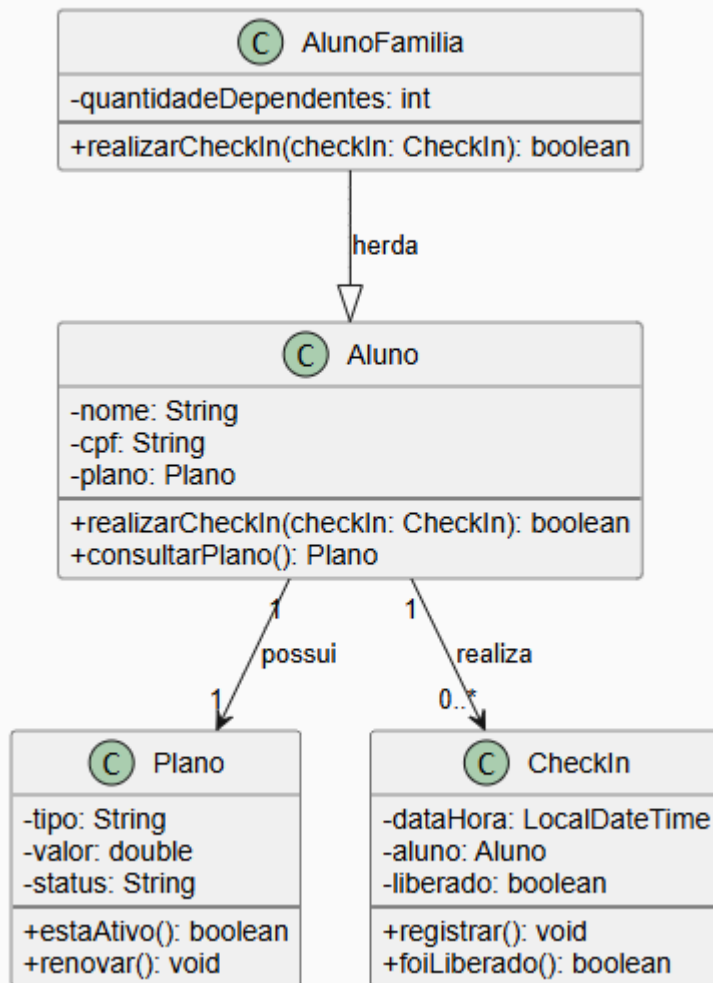
****Operações:****

```
* `registrar(): void`  
* `foiLiberado(): boolean`
```

3. Diagrama de Classes

O modelo das classes pode ser representado da seguinte forma:

Diagrama de Classes - Academia



Relacionamentos

- * Um `Aluno` possui um `Plano` ativo.
- * Um `Aluno` pode possuir vários registros de `CheckIn`.
- * Cada `CheckIn` está associado a um `Aluno`.

Em termos de multiplicidade:

```

```text
Aluno 1 ————— 1 Plano

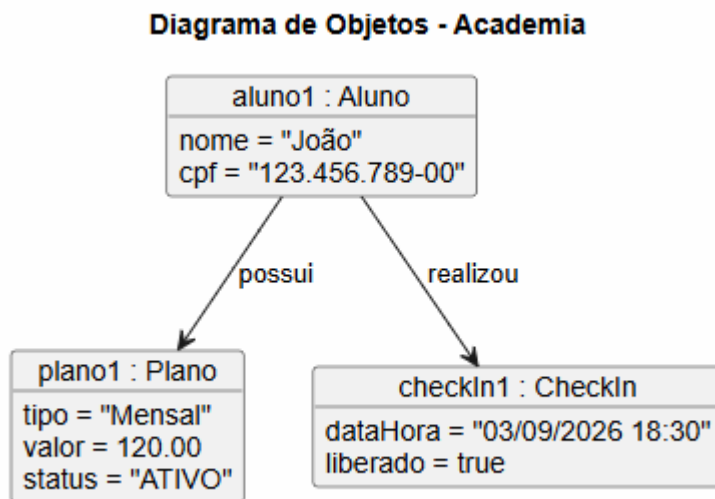
Aluno 1 ————— 0..* CheckIn
```

```

4. Diagrama de Objetos

Enquanto o diagrama de classes apresenta a estrutura geral do sistema, o diagrama de objetos apresenta exemplos concretos de objetos existentes em determinado momento.

Considerando um aluno chamado João, com um plano mensal ativo e um check-in realizado:



Neste exemplo:

- * ``aluno1`` é uma instância da classe ``Aluno``;
- * ``plano1`` é uma instância da classe ``Plano``;
- * ``checkIn1`` é uma instância da classe ``CheckIn``.

5. Mensagem entre objetos

Uma interação possível entre os objetos do sistema ocorre quando um aluno tenta realizar um check-in.

A mensagem pode ser representada por:

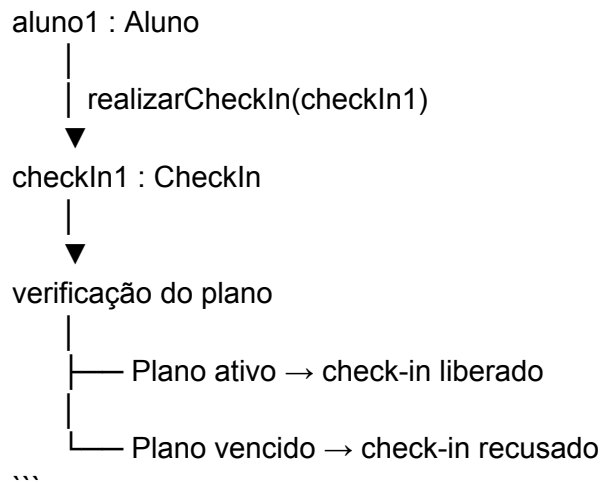
```
```text
aluno1.realizarCheckIn(checkIn1)
```
```

Nesse caso, o objeto ``aluno1`` solicita a realização do check-in utilizando o objeto ``checkIn1``.

Durante essa operação, o sistema deve verificar se o plano associado ao aluno está ativo. Caso esteja, o check-in pode ser liberado. Caso o plano esteja vencido ou inativo, o check-in deverá ser recusado.

Exemplo conceitual:

```
```text
```



Essa interação demonstra a comunicação entre objetos dentro do sistema.

---

## # 6. Os quatro pilares da Orientação a Objetos

### ## 6.1 Abstração

A abstração está presente na criação das classes `Aluno`, `Plano` e `CheckIn`.

Cada classe representa uma entidade relevante do domínio da academia e contém somente as informações e comportamentos necessários para representar aquela entidade dentro do sistema.

Por exemplo, `Aluno` possui informações como nome, CPF e plano, além de comportamentos relacionados ao seu uso dentro da academia.

---

### ## 6.2 Encapsulamento

O encapsulamento aparece na definição dos atributos como privados.

Por exemplo:

```

```text
Aluno
-----
- nome: String
- cpf: String
- plano: Plano
```

```

O símbolo `-` representa atributos privados no modelo UML.

Dessa maneira, os dados internos do objeto não devem ser modificados diretamente por outras partes do sistema. As alterações devem ocorrer por meio dos comportamentos definidos pelas próprias classes.

Isso também permite proteger regras do sistema, como impedir que um plano receba um valor inválido.

---

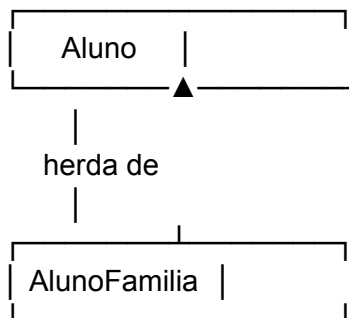
## ## 6.3 Herança

O projeto também possui um tipo especial de aluno: o aluno com plano família.

Como `AlunoFamilia` representa um tipo de `Aluno`, podemos utilizar herança para reaproveitar as características existentes na classe `Aluno`.

O relacionamento pode ser representado assim:

```text



```

A classe `AlunoFamilia` pode reutilizar os atributos e operações de `Aluno` e acrescentar características específicas, como a quantidade de dependentes.

Exemplo:

```text

Aluno

- nome

- cpf

- plano



AlunoFamilia

- quantidadeDependentes

```

---

## ## 6.4 Polimorfismo

O polimorfismo pode ser utilizado quando diferentes tipos de alunos possuem a mesma operação, mas podem apresentar comportamentos específicos.

Por exemplo, a operação:

```
```text
realizarCheckIn()
```
```

pode existir na classe `Aluno` e ser redefinida em uma classe especializada, como `AlunoFamilia`, caso futuramente existam regras específicas para esse tipo de aluno.

Assim, o sistema pode trabalhar com uma referência do tipo `Aluno`, enquanto o comportamento executado depende do tipo real do objeto.

Isso evita a necessidade de criar diversos blocos condicionais (`if/else`) para identificar manualmente cada tipo de aluno.

---

## # 7. Resumo dos conceitos aplicados

| Pilar                 | Aplicação no projeto                                                                        |
|-----------------------|---------------------------------------------------------------------------------------------|
| -----                 | -----                                                                                       |
| <b>Abstração</b>      | Classes `Aluno`, `Plano` e `CheckIn` representam entidades relevantes do domínio.           |
| <b>Encapsulamento</b> | Atributos privados e acesso controlado pelos comportamentos das classes.                    |
| <b>Herança</b>        | `AlunoFamilia` especializa a classe `Aluno`.                                                |
| <b>Polimorfismo</b>   | Tipos diferentes de aluno podem possuir comportamentos específicos para uma mesma operação. |

---

## # 8. Conclusão

O modelo desenvolvido representa uma primeira versão do sistema de academia, contemplando alunos, planos e registros de frequência.

A modelagem permite representar as principais entidades do domínio e seus relacionamentos, além de demonstrar a comunicação entre objetos.

Também foram aplicados os quatro pilares da Orientação a Objetos: abstração, encapsulamento, herança e polimorfismo.

O modelo poderá ser expandido nas próximas etapas do projeto, incluindo regras mais completas para o ciclo de vida dos planos, formas de pagamento, renovação e controle de acesso à academia.